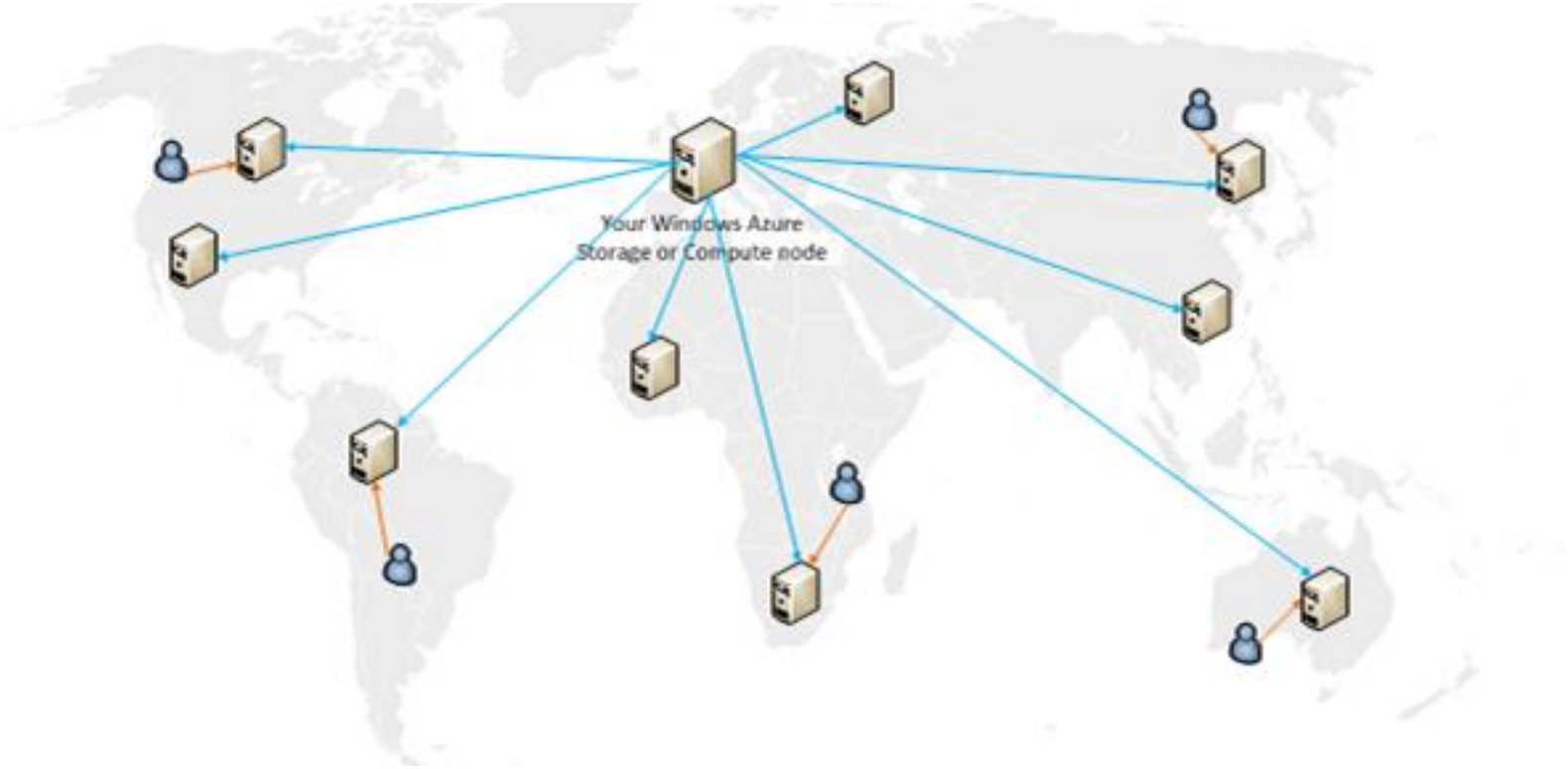
# Scaling Static Pages




Slides by Andrew DeOrio

# Scaling road map

- Today: Scaling static pages
- Next: Scaling dynamic pages
- Later: Scaling storage
  - Database
  - Media

# Agenda

- **Cloud**
- Scaling static pages
- Domain name system (DNS)
  - Resolution
  - Scaling and security
- Content delivery networks (CDNs)

# Motivation

- You built a web app that works for one user
- Runs locally on your laptop

- Question: How do we support millions of users?
- Answer: Move to a scalable *production environment*

# Development vs. production

- *Development*
  - *For building and debugging*
  - Runs on one machine
  - Example: `flask run`

- *Production*
  - For real users
  - Runs on many machines
  - Specialized and scalable

- Problem: Where do these computers live?
- Solution: Buy servers

# On-premises servers

- Buy your own servers
- Build your own datacenter
- AKA "private cloud"
- AKA "on-premises"

# Datacenter

- Large facility with thousands of servers

- Requires:
  - Power
  - Colling
  - High bandwidth

- Lots of computers means lots of computer failures

# On-premises to IaaS

- Problem:
  - Managing physical servers is difficult
  - Power, cooling, failures, maintenance
- Solution:
  - Rent computers instead of owning them

# Infrastructure as a Service (IaaS)

- *Infrastructure as a Service (IaaS)*
  - Rent a computer in somebody else's datacenter

- Benefits
  - No hardware management
  - Built-in scaling
  - High availability

- Examples:
  - AWS EC2
  - Google Compute Engine
  - Microsoft Azure Virtual Machines

# IaaS to PaaS

- Problem: Managing software on many rented computers is complex
  - Security updates, complex configuration
- Solution: Rent computers with software already installed and configured

# Platform as a Service (PaaS)

- *Platform as a Service (PaaS):* Rent a computer in somebody else's data center
  - They manage and configure the software
  - You manage your application

- Example: servers with database already installed
  - AWS Relational Database Service (RDS)
  - Google Cloud SQL
  - Microsoft Azure SQL

# PaaS to SaaS

- Problem: Sometimes you don’t want to build or mangage an app.

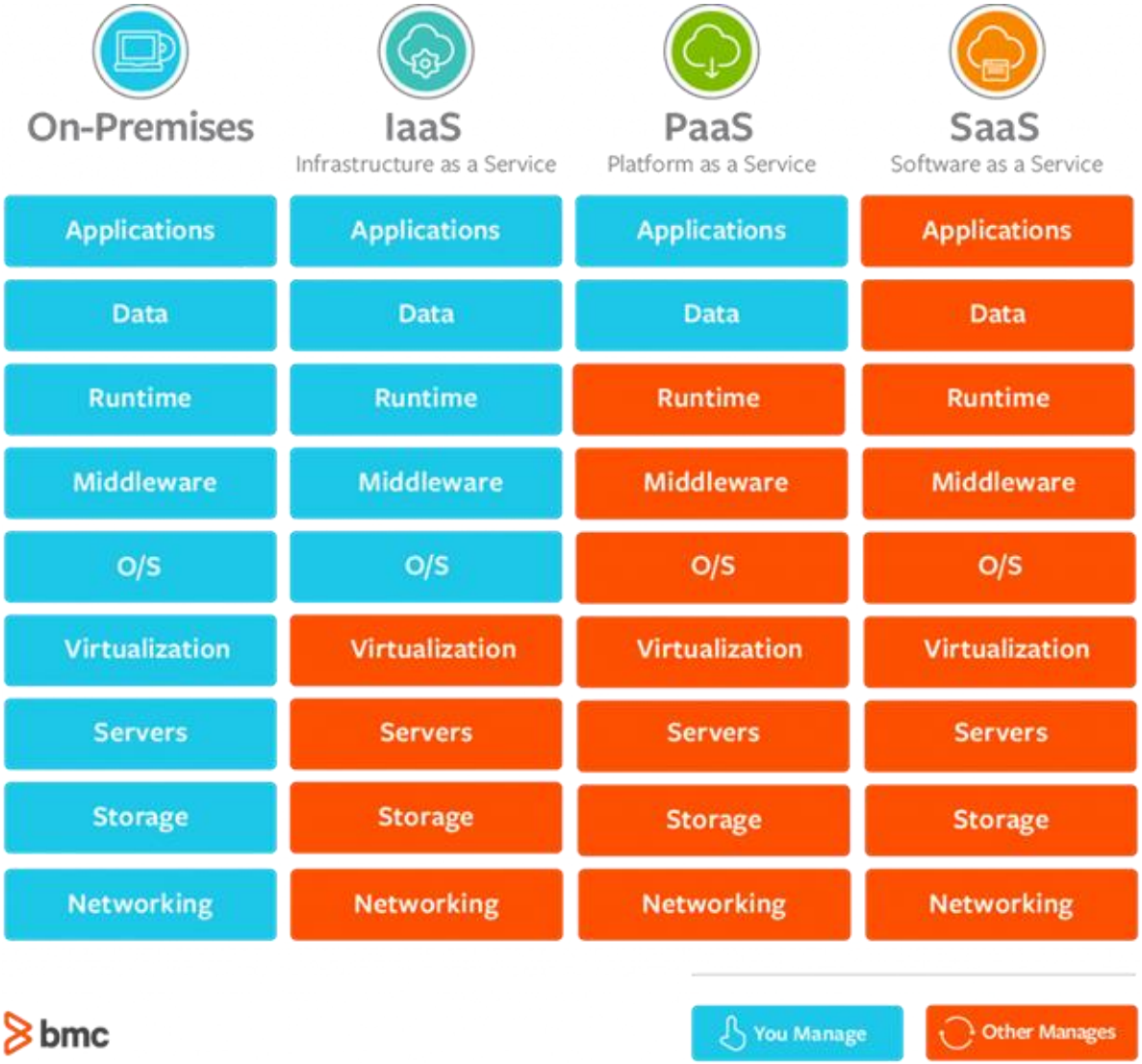
- Solution: Rent a fully managed web application running on somebody else's computers

# Software as a Service (SaaS)

- *Software as a Service (SaaS):* Fully managed applications
  - Zero effort
  - No infrastructure of maintenance
  - Ready to use

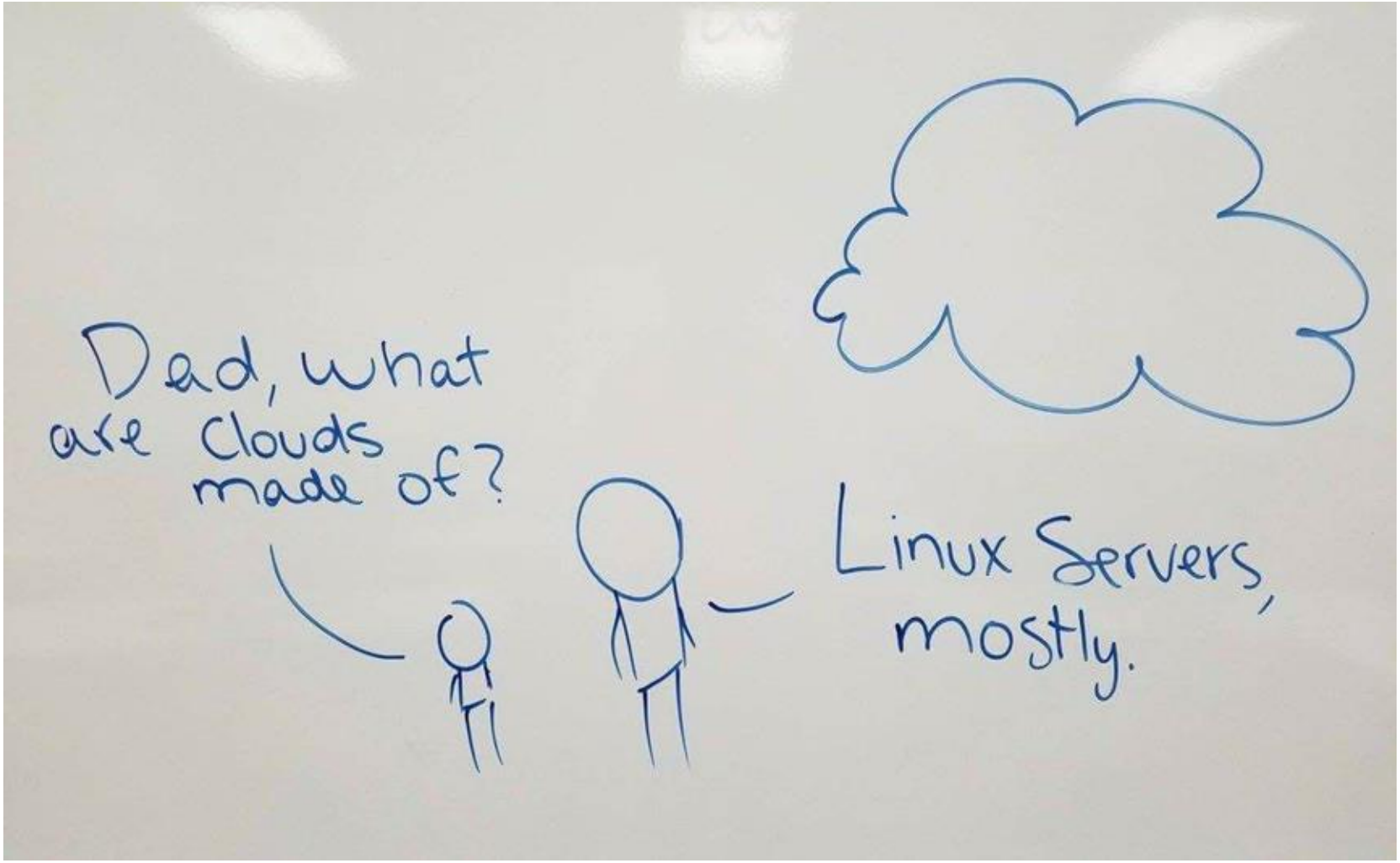
- Examples:
  - Gmail
  - Dropbox
  - GitHub

# IaaS vs. PaaS vs. SaaS




https://www.bmc.com/blogs/saas-vs-paas-vs-iaas-whats-the-difference-and-how-to-choose/

# Cloud memes



# Web apps in the cloud

- Problem: How do I run my web app in the cloud?

- Scale one thing at time
  - Static pages
  - Server-side dynamic pages
  - Client-side dynamic pages
  - Database storage
  - Media uploads storage

# Agenda

- Cloud
- **Scaling static pages**
- Domain name system (DNS)
  - Resolution
  - Scaling and security
- Content delivery networks (CDNs)

# Static pages

- Static pages: a request returns a copy of a file

- Problem: How do we serve copies to many clients at the same time?
- Solution: many servers

- Problem: It takes a long time for a request/response to traverse the global internet
- Solution: servers in different geographic locations

# Review: IP Address

- Every device on the Internet has an IP address
  - 172.217.5.14
- But humans use domain names
  - Example: umich.edu
- DNS translates domain names to IP addresses

# DNS directs to different regions

- Deploy servers in different geographic regions
- Use DNS to route users to nearby servers

- Question: How does DNS work?

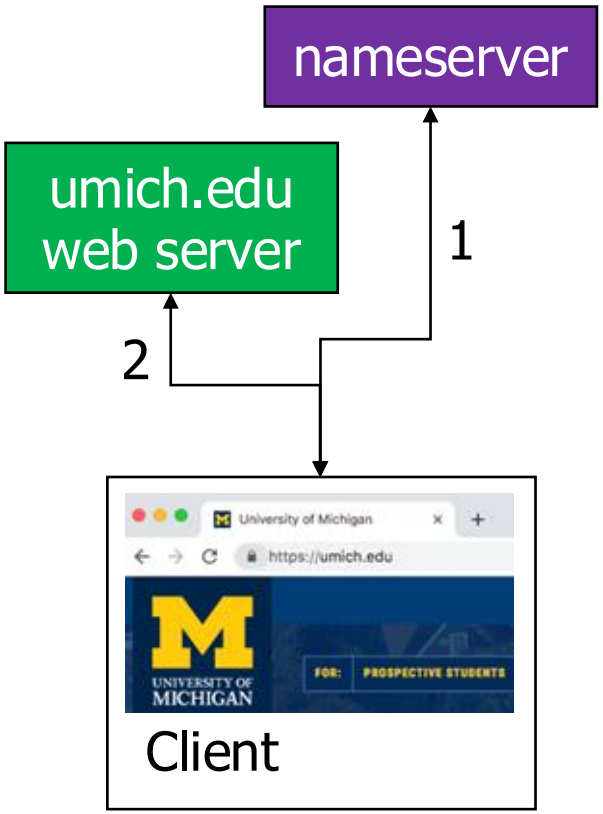
# Agenda

- Scaling static pages intro
- **Domain name system (DNS)**
  - Namespace
  - Resolution
  - Scaling and security
- Content delivery networks (CDNs)

# Domain Name System (DNS)

- Domain Name System (DNS) Translates domain names to IP addresses
- Example: `umich.edu -> 141.211.243.251`
- Basically a database of (hostname, IP) pairs

# DNS example




**Nameserver**
provides IP of
**umich.edu**

# Agenda

- Cloud
- Scaling static pages intro
- Domain name system (DNS)
  - **Resolution**
  - Scaling and security
- Content delivery networks (CDNs)

# DNS Resolution

- Content of DNS consists of *resource records*
  - Host -> IP

- Most DNS activity
  - Client requests to authority servers
  - Server responds with resource record

# Where do I get a DNS server?

- IP addresses are commonly assigned automatically
  - When you connect to a WIFI or wired network

- DHCP: Dynamic Host Configuration Protocol
  - Assigns an IP
  - Provides DNS servers

# Resolution with more detail

- In the previous example, our DNS server had a cached copy of the IP address for `umich.edu`
- What if it didn't have a cached copy?
- Ask the authority servers

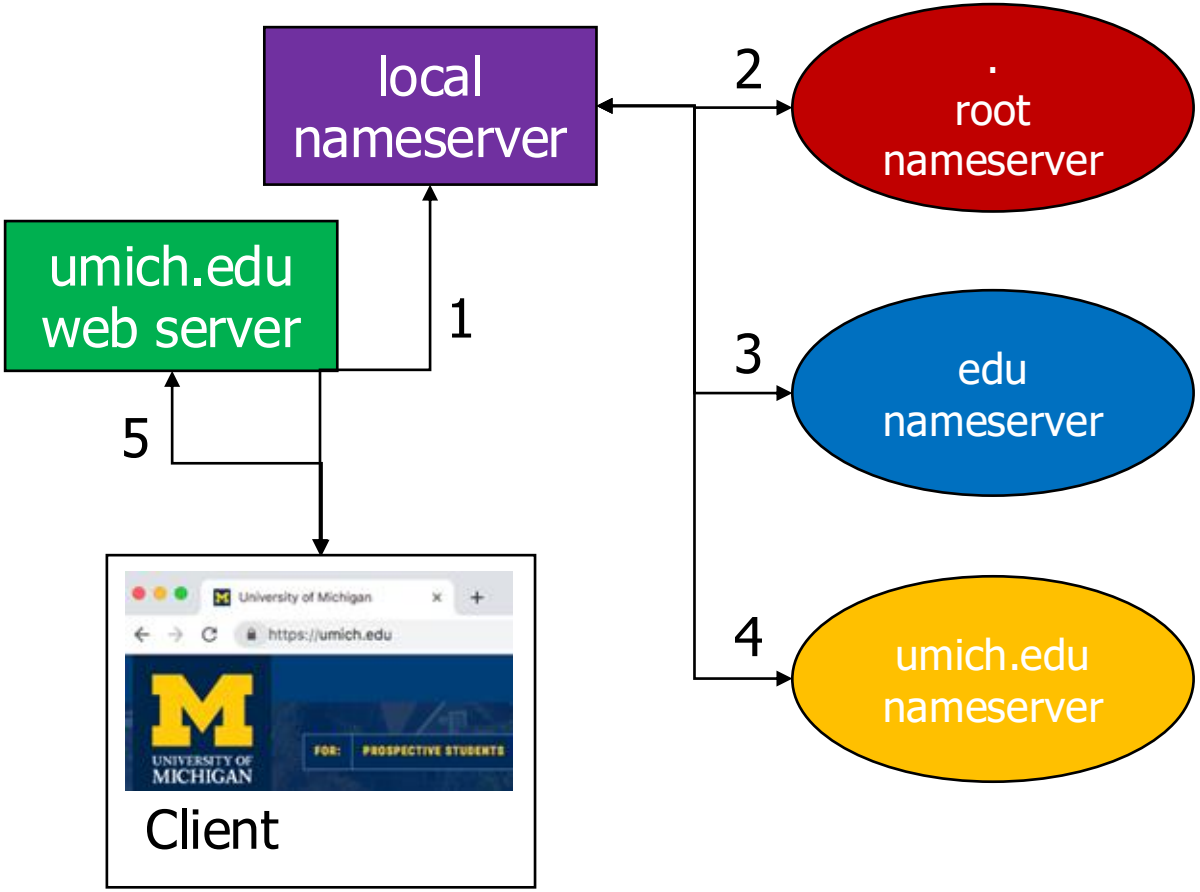
# DNS Resolution Steps

- DNS resolution steps to open umich.edu

- Ask root server where is .edu
- Ask .edu server where is umich.edu
- Ask umich.edu server to get IP

- Result: Client receives IP of umich.edu

# Resolution



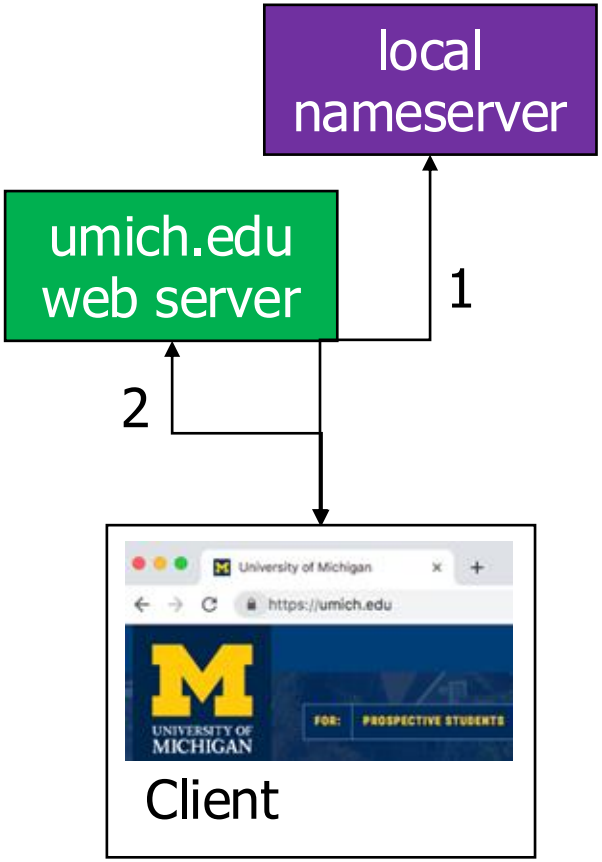

**local nameserver** provides IP of **root nameserver**

**root nameserver** provides IP of **edu nameserver**

**edu nameserver** provides IP of **umich.edu nameserver**

**umich.edu nameserver** provides IP of **umich.edu web server**

# Resolution with caching




**local nameserver** provides IP of **umich.edu web server** because it's in the cache

# DNS Caching

- DNS Caching: DNS results are cached for speed
- Great!  But when data changes?
  - TTL (Time-To-Live): How long a record stays valid

# Root nameservers

- Responsible for locating the top level domain name servers (.com, .net, .org, …)
- 13 root name servers in world
  - Their locations are hard-coded in resolving DNS servers
  - Due to caching, involved in few queries

# Agenda

- Scaling static pages intro
- Domain name system (DNS)
  - Namespace
  - Resolution
  - **Scaling and security**
- Content delivery networks (CDNs)

# DNS scale

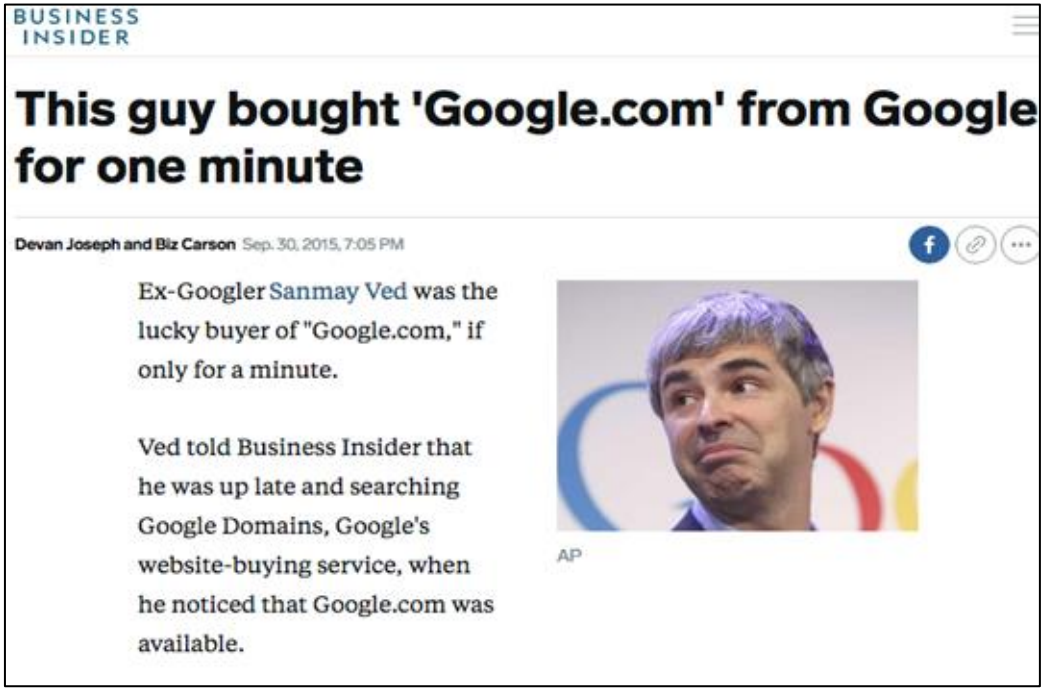
- DNS operates at massive scale
  - Used in nearly every internet request
- Impacts
  - Computers
  - Phones
  - Internet of Things (IoT) devices
  - Backend services

# DNS as a security vulnerability

- DNS is an extreme security vulnerability
- What if someone can remap google.com?
- What about just denial-of-service?
- Terrorism

# What if someone can remap google.com?

- Domain name registrations have an expiration date
- Google's expired one time
- Registrar reversed it



BUSINESS INSIDER

**This guy bought 'Google.com' from Google for one minute**

Devan Joseph and Biz Carson Sep. 30, 2015, 7:05 PM

Ex-Googler Sanmay Ved was the lucky buyer of "Google.com," if only for a minute.

Ved told Business Insider that he was up late and searching Google Domains, Google's website-buying service, when he noticed that Google.com was available.

AP



https://www.businessinsider.com/this-guy-bought-googlecom-from-google-for-one-minute-2015-9

# DNS cache poisoning

- Bad info is inserted into a DNS server and cached
- Both inadvertent and malicious
- How could one attach CSE's DNS server?

# DNS cache poisoning

- DNS cache poisoning is one of the ways that China's Great Firewall works
- Just direct requests to twitter.com to the wrong IP

# DNS security remedies

- If you use HTTPS/SSL/TSL, problem mitigated
- DNSSEC is a security extension to DNS
  - Cryptographically signed DNS entries
  - Asymmetric cryptography

# Agenda

- Scaling static pages intro
- Domain name system (DNS)
  - Namespace
  - Resolution
  - Scaling and security
- **Content delivery networks (CDNs)**

# Scaling static pages solution

- So far, we've talked about how to convert a hostname into the IP address of a server
- How do we return the IP address of a nearby server?

# Content delivery networks

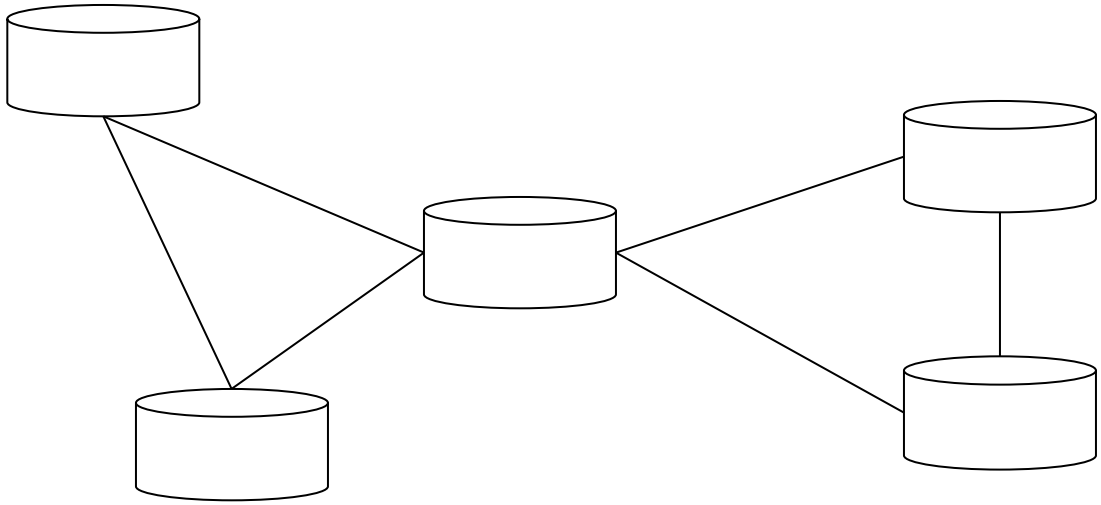
- A content delivery network (CDN) stores static files at many locations throughout the world and serves nearby clients
- Static files: HTML, images, videos
- Large read-only data stored close to client
- Reduce latency to faraway datacenter
- Reduce bandwidth costs by sending data only a short distance

# Akamai

- The first content-delivery network
  - You might think the image is from CNN, but Akamai is serving it
  - CNN pays Akamai to do so
- Built on top of a giant DNS hack

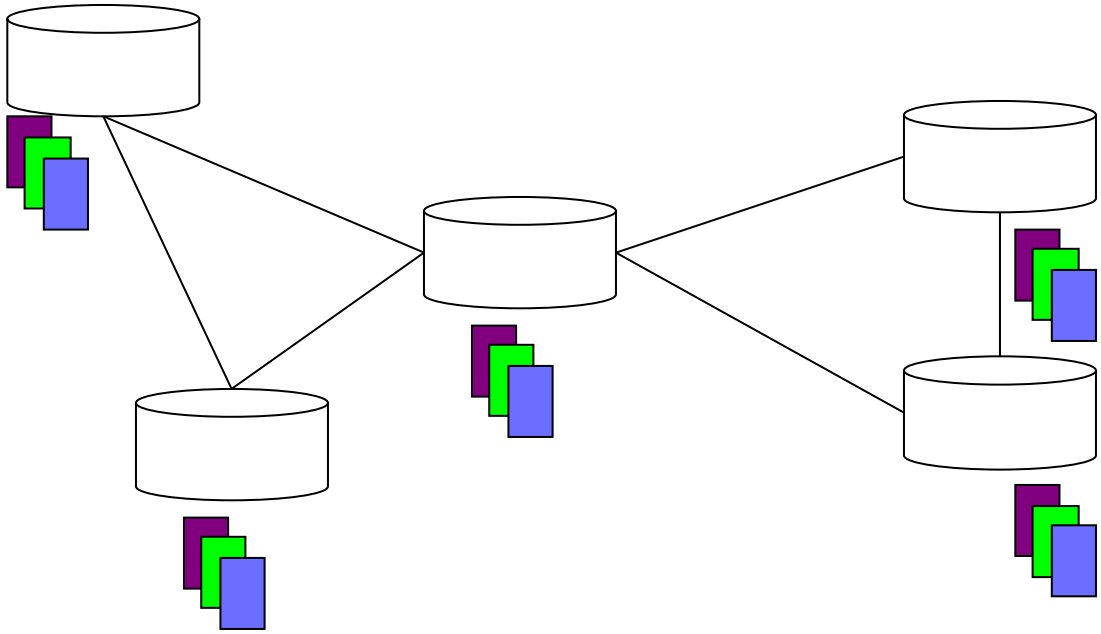
# How it works: step 1

- Akamai places servers across the world



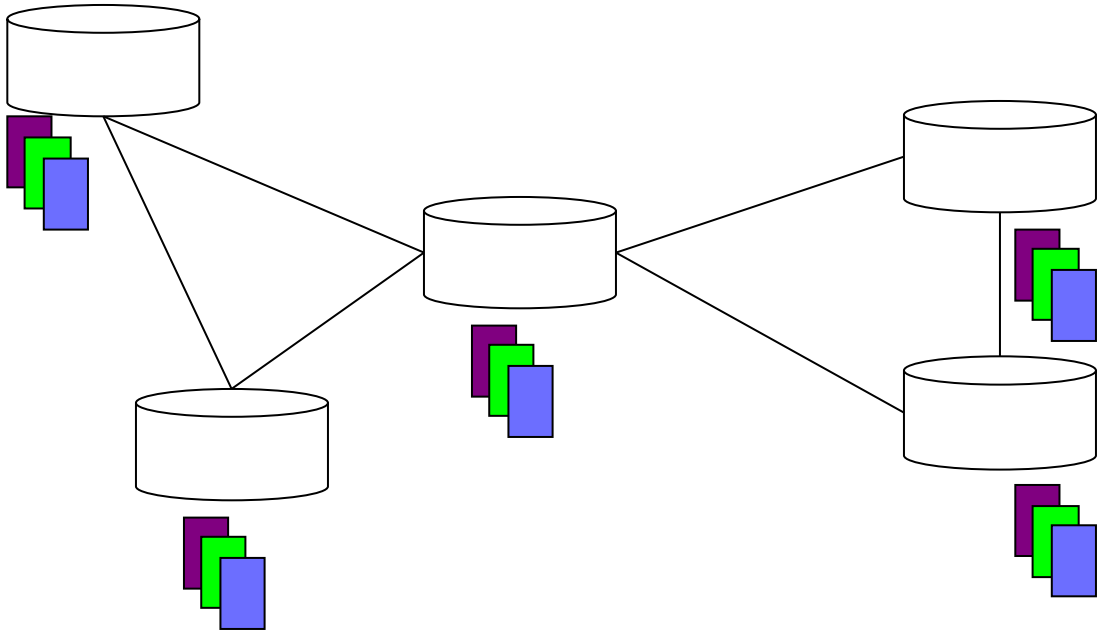
# How it works: step 2

- Akamai gets client data
  - Especially images, videos
- Copies to all nodes in network



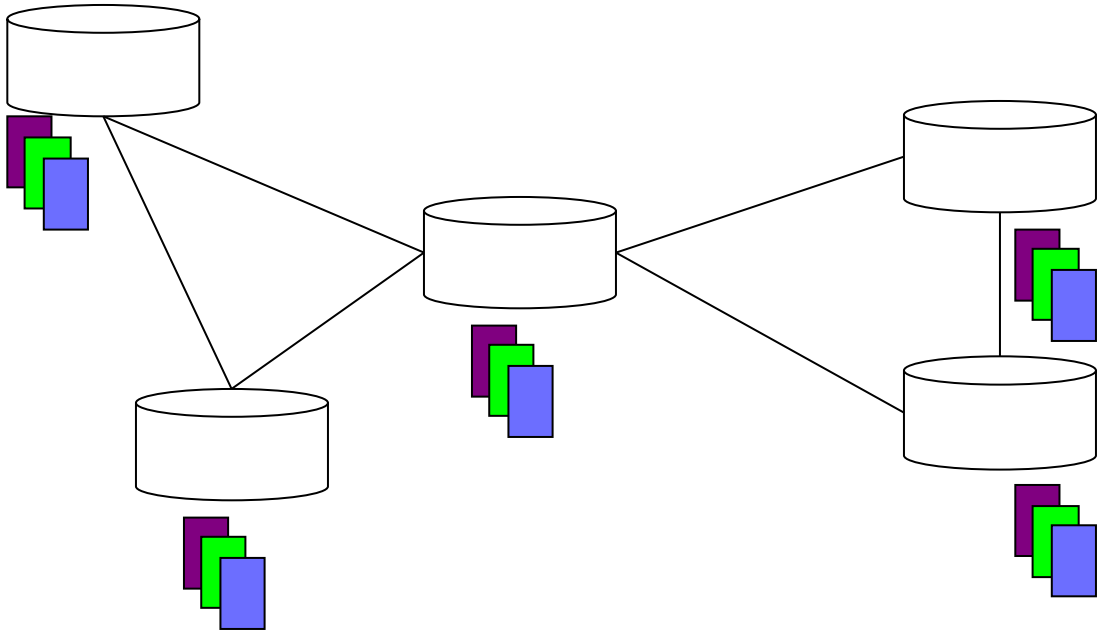
# How it works: step 3

- How does client find nearby data?
  - Network conditions can change



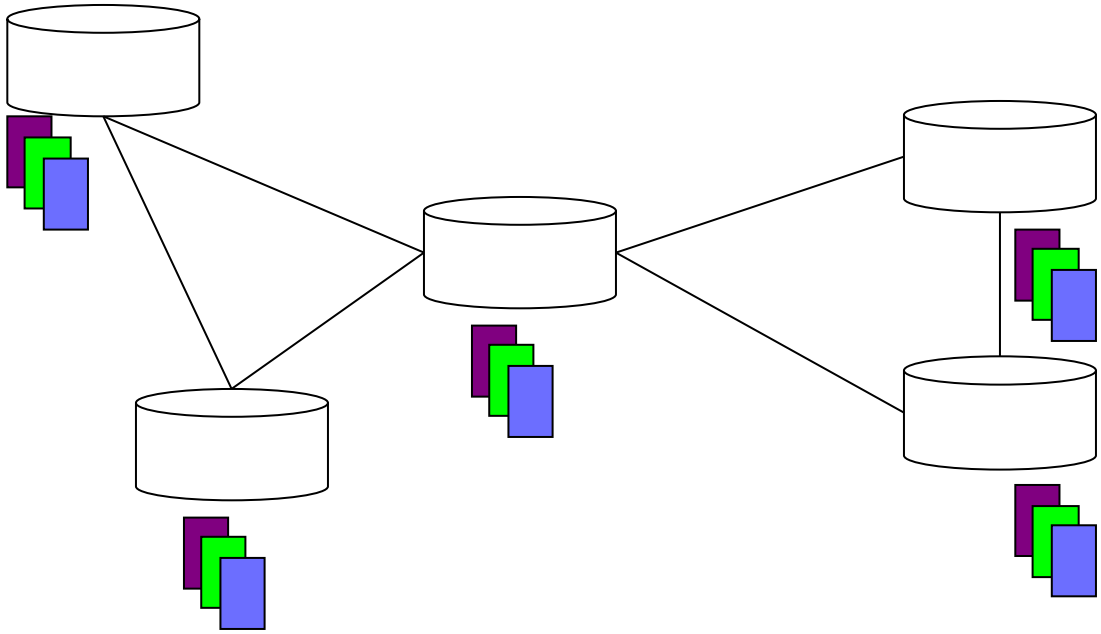
# How it works: step 3

- Website rewrites their URLs to uses CDN URLs
  - http://cnn.com/… => http://akadns.akamai.net/….



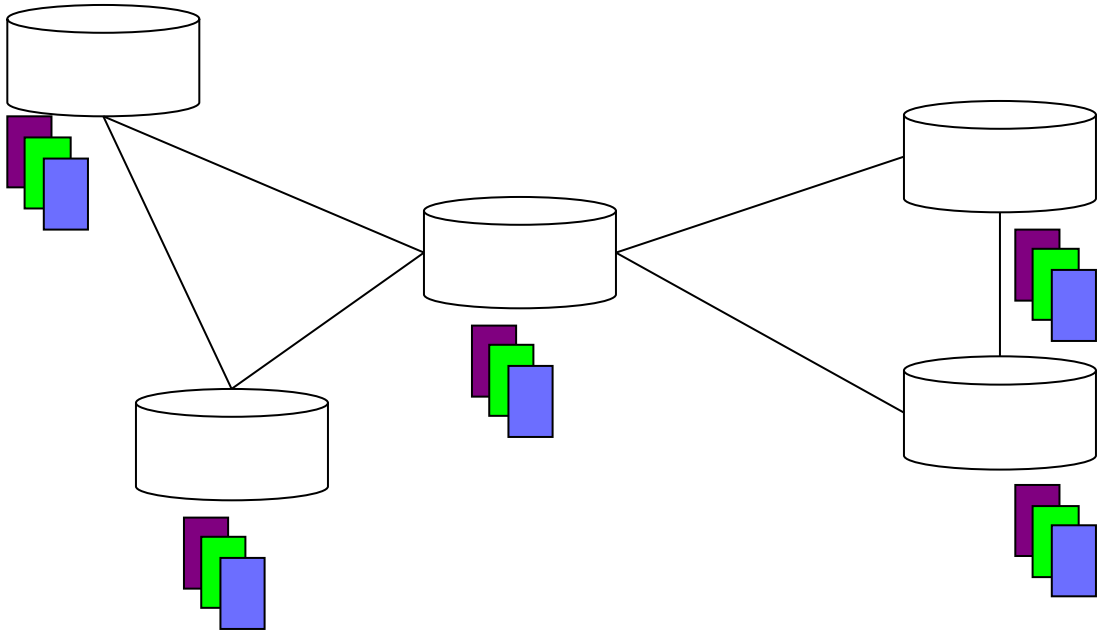
# How it works: step 3

- URL contains Akamai-controlled name
  - Specialized Akamai DNS server resolves name to nearby server; tiny TTL



# How it works: step 3

- Resolution strongly dependent on
  - Location of client; network conditions; load on Akamai servers; traffic estimation errors



# Akamai's scale

- Akamai has 170k servers
- Serves ~15%-30% of all web traffic

# CDN summary

- Custom DNS servers dynamically direct clients to best caching servers
- Static content
- Reduces latency for clients fetching content
- Reduces load on datacenter servers

# Summary

- Cloud: "Where do we put all the computers"
- Domain name system (DNS): Distributed system for translating hostname to IP
  - Resolve a domain name to an IP address
  - `umich.edu -> 141.211.243.251`
- Content delivery network (CDN): Distributed system for serving static pages
  - Duplicate static data
  - Direct clients to nearby servers
  - Use modified DNS

# Scaling road map

- Today: Scaling static pages
- Next time: Scaling dynamic pages
- After that: Scaling storage
  - Database
  - Media uploads